Assignment 2

Implementation of Auto-Scaling and Security in Google Cloud Platform

Pradeepika Nori (B23CS1047)
Indian Institute of Technology Jodhpur

March 1, 2026

1 Objective

The objective of this assignment is to design and deploy a scalable and secure virtual machine infrastructure using a public cloud platform. The implementation focuses on three major aspects:

1. Dynamic resource provisioning through CPU-based auto-scaling.
2. Secure access management using Identity and Access Management (IAM).
3. Network protection through firewall rule configuration.

Google Cloud Platform (GCP) was selected to demonstrate these concepts.

2 Documentation Links

Source Code Repository:
https://github.com/b23cs1047/GCP-AutoScaling-Security

Video Demonstration:
Project Demonstration Video

3 Cloud Platform Used

The implementation was carried out using **Google Cloud Platform (GCP)**, specifically the **Compute Engine** service. Compute Engine provides Infrastructure-as-a-Service (IaaS) capabilities that allow users to deploy configurable virtual machines while supporting scalability, automation, and security management.

4 Implementation Methodology

4.1 Project Initialization

A new cloud project was created in GCP, after which the Compute Engine API was enabled to allow creation and management of virtual machines and related infrastructure components.

4.2 Instance Template Configuration

An Instance Template was created to define a standardized configuration for all virtual machine instances. The template ensures consistency whenever new instances are created during auto-scaling.

The configuration included:

- Machine Type: `e2-micro`
- Operating System: Ubuntu Linux
- HTTP Traffic Enabled
- Automated startup script execution

4.3 Startup Script Automation

A startup script was attached to the instance template to automate server configuration. Whenever a new instance is launched, the script installs and starts an Nginx web server and dynamically generates a webpage displaying instance metadata such as hostname, IP address, and creation time.

This automation removes manual configuration and demonstrates infrastructure provisioning at scale.

4.4 Managed Instance Group (MIG)

A Managed Instance Group was created using the instance template.

The group maintains identical VM replicas and ensures service availability by automatically replacing failed instances. The following scaling limits were configured:

- Minimum Instances: 1
- Maximum Instances: 3

4.5 Auto-Scaling Policy

Auto-scaling was enabled based on average CPU utilization.

- Scaling Metric: CPU Utilization
- Target Utilization: 60%

When CPU usage exceeds the threshold, additional instances are provisioned automatically. When workload decreases, surplus instances are removed, ensuring efficient resource utilization and cost optimization.

5 Auto-Scaling Validation

To verify the functionality of auto-scaling, artificial computational load was generated through SSH access.

5.1 Procedure

```
sudo apt install stress
stress --cpu 2 --timeout 300
```

5.2 Observation

During execution, CPU utilization crossed the defined threshold, triggering automatic creation of additional VM instances within the Managed Instance Group. After workload reduction, instances were scaled down automatically, confirming correct operation of the scaling policy.

6 Security Implementation

6.1 Identity and Access Management (IAM)

IAM was configured to enforce controlled administrative access.

A secondary user account was assigned the **Compute Viewer** role. This role allows monitoring and viewing of resources while preventing modification or deletion actions.

This configuration demonstrates the *Principle of Least Privilege*, ensuring users receive only necessary permissions.

6.2 Firewall Configuration

Network security was enforced using VPC firewall rules.

Allowed Traffic

- HTTP access on Port 80
- Secure Shell (SSH) access on Port 22

Restricted Traffic

An explicit deny rule was implemented:

- Rule Name: `deny-unnecessary-ports`
- Direction: Ingress
- Blocked Port: 8080

The deny rule minimizes exposure to unused services and strengthens overall security posture.

7 Architecture Design

The overall system architecture integrates scalability and security mechanisms within a unified cloud environment. The architecture consists of an access layer, a security layer, and an auto-scaling compute layer governed by IAM policies.

The interaction between these components enables seamless coordination between scalability and security within the cloud environment. Internet users initiate requests that first traverse the Virtual Private Cloud (VPC) network, where firewall policies enforce controlled

access by allowing only required communication channels. This ensures that the system remains protected from unauthorized traffic while maintaining availability for legitimate users.

Once traffic passes the network security layer, requests are handled by the Managed Instance Group, which dynamically manages multiple virtual machine instances created from a common instance template. Each instance is automatically configured during initialization through a startup script that installs and launches the Nginx web server, ensuring consistent deployment across all replicas.

The CPU-based auto-scaling mechanism continuously monitors resource utilization across running instances. During periods of increased workload, additional instances are provisioned automatically to maintain performance and responsiveness. When demand decreases, excess instances are removed, enabling efficient resource utilization and cost optimization.

Administrative operations are governed independently through IAM role-based access control. By assigning restricted roles to users, the system follows the principle of least privilege, preventing accidental or unauthorized configuration changes. Together, these components form a robust cloud architecture capable of adapting to varying workloads while maintaining strong security controls.

Furthermore, the architectural design promotes reliability and fault tolerance by leveraging the self-healing capabilities of the Managed Instance Group. If an instance becomes unhealthy or fails unexpectedly, the system automatically replaces it using the predefined instance template, ensuring uninterrupted service availability. This automated recovery mechanism minimizes downtime and reduces the need for manual intervention. By combining elasticity, automated configuration, and layered security controls, the architecture reflects modern cloud computing principles where scalability, availability, and protection are integrated as foundational design elements rather than afterthoughts.

The primary components include:

- Internet Users
- VPC Firewall Rules (Allow and Deny Policies)
- Managed Instance Group (MIG)
- CPU-Based Auto-Scaling Policy
- Nginx Web Server Instances
- IAM Role-Based Access Control

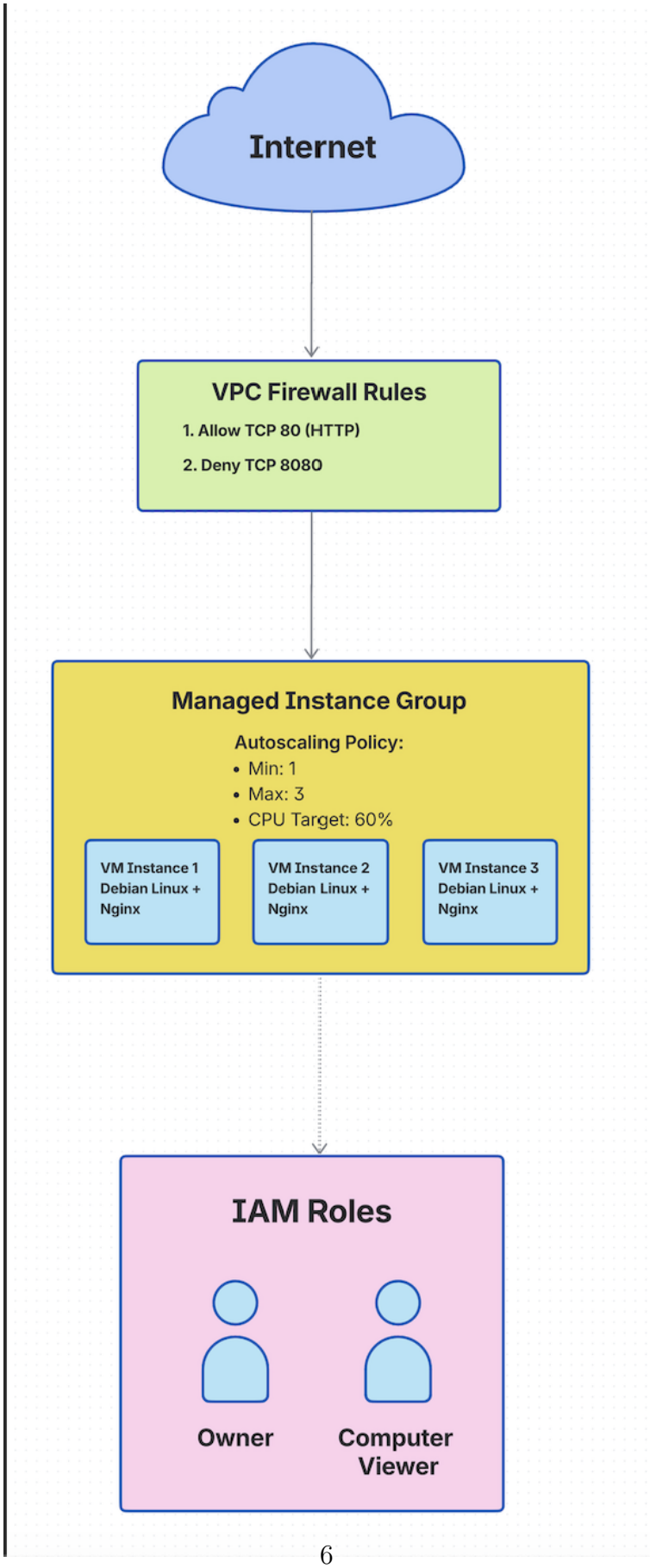
Internet
VPC Firewall Rules
1. Allow TCP 80 (HTTP)
2. Deny TCP 8080
Managed Instance Group
Autoscaling Policy:
• Min: 1
• Max: 3
• CPU Target: 60%
VM Instance 1
Debian Linux +
Nginx
VM Instance 2
Debian Linux +
Nginx
VM Instance 3
Debian Linux +
Nginx
IAM Roles
Owner
Computer
Viewer

Figure: Architecture of Auto-Scaling and Security Implementation in GCP

Incoming user traffic is filtered through firewall rules before reaching the managed instance group. The system dynamically provisions new virtual machine instances when CPU utilization exceeds the defined threshold. Administrative access is controlled using IAM policies that enforce least-privilege permissions.

8 Conclusion

This assignment successfully demonstrated deployment of a scalable and secure cloud infrastructure using Google Cloud Platform. CPU-driven auto-scaling enabled adaptive resource allocation, while IAM roles and firewall rules ensured controlled and secure system access.

The implementation reflects practical cloud engineering principles involving automation, elasticity, and security-aware infrastructure design.